Longitude — everything used in this project

Generated 29 September 2026 · Longitude · Next.js 16 · React 19 · CesiumJS · PostgreSQL

This guide lists every technology, library, outside service, API route, database table, technique and file in Longitude, and explains what each one is for. It pairs with the README, which covers setup and how the site behaves. A glossary at the end explains the technical terms.

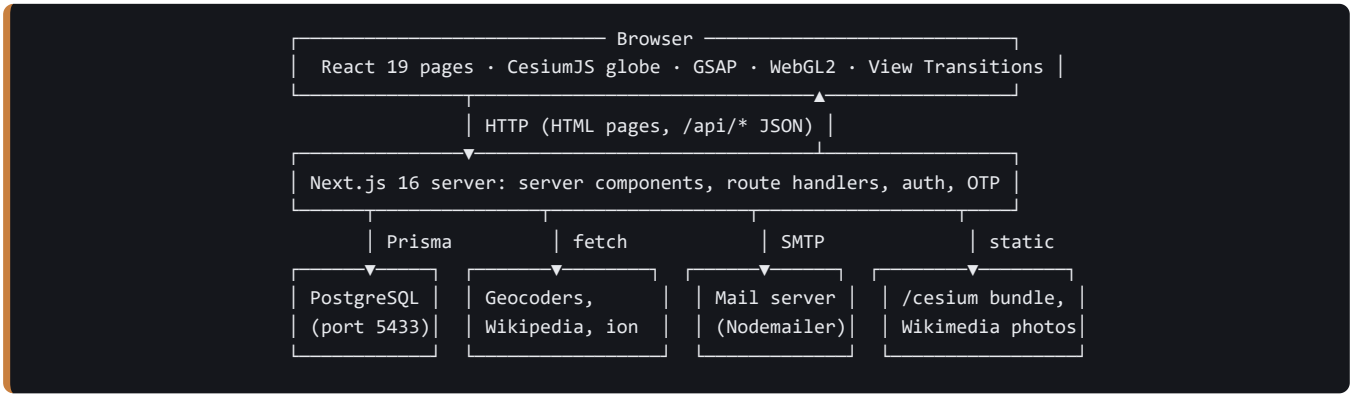
Contents

1. The big picture
2. Technology stack
3. Outside services and data sources
4. Architecture
5. Pages and what's on them
6. API reference
7. Database
8. Accounts, sessions and the email code
9. 3D, graphics and animation
10. Design system
11. Configuration
12. Scripts and everyday workflows
13. File-by-file map
14. Glossary

---

1. The big picture

Longitude is a **full-stack web app**: one Next.js project holds both the website (what runs in the browser) and the server (pages rendered on the server, plus a JSON API). Data lives in **PostgreSQL**, reached through **Prisma**. The 3D Earth is **CesiumJS** streaming real terrain, imagery and photorealistic 3D cities from **Cesium ion**. The motion is **GSAP** plus several hand-written **WebGL2** renderers. Accounts use a password and a **6-digit code sent by email**.



---

2. Technology stack

Versions are the ones installed in `node_modules` at the time of writing.

Framework and language

Technology	Version	What it is	What Longitude uses it for
Node.js	20+ (built on 24)	JavaScript runtime	Runs Next.js, the database scripts, the seed and the mailer. Its built-in `node:crypto` does all password hashing, code generation and signing.
Next.js	16.3.6	React framework (App Router, Turbopack)	Routing by folders in `src/app`, server-rendered pages, API route handlers, font loading (`next/font`), image domains, static generation of every destination and tour page (`generateStaticParams`) refreshed every 5 minutes (`revalidate = 300`).
React	19.2.8	UI library	Every component. Server Components render data-heavy pages on the server; Client Components (`"use client"`) handle the globe, animation and forms.
TypeScript	5.9.3	Typed JavaScript	All source code (`.ts`, `.tsx`), checked with `npm run typecheck`.

Styling and type

Technology	Version	What Longitude uses it for
Tailwind CSS	4.3.3 (with `@tailwindcss/postcss`)	Utility classes in layouts and the design tokens (`--color-*`, `--font-*`) that Tailwind exposes as classes such as `bg-space`, `text-signal`.
Hand-written CSS	—	`src/app/globals.css` holds the design system and every signature section (boarding passes, timetable, prints, key card, code step…). Uses modern CSS: `color-mix()`, `clip-path`, `mask`, `backdrop-filter`, `@keyframes`, `prefers-reduced-motion` and `(hover: hover)` media queries, `inert`.
Geist, Geist Mono	via `next/font/google`	The sans-serif for text and headings; the monospace for HUD labels, codes, prices and numbers.
Instrument Serif (italic)	via `next/font/google`	Editorial accents only: the city name in headings ("frames from *London*.").
Phosphor Icons	`@phosphor-icons/react` 2.1.10	Every icon (arrows, airplane, check, shuffle, pause/play…).

Motion and graphics

Technology	Version	What Longitude uses it for
GSAP	3.15.0	The animation engine. Timelines, tweens, `quickTo` for cursor-follow tilt, `matchMedia` for reduced-motion variants. Registered once in `src/lib/gsap.ts`.
GSAP **ScrollTrigger**	plugin	Scroll-driven motion: the home journey's scroll → camera mapping, pinned sections (tours route, the prints), reveals as sections enter.
GSAP **ScrollToPlugin**	plugin	Smooth scrolling to "#book", the picked timetable row, etc.
GSAP **SplitText**	plugin	Registered and ready, but not used yet: headline reveals split their words in the React markup instead.
GSAP **Flip**	plugin	The timetable's re-sort: rows glide from their old places to their new ones.
GSAP **CustomEase** + **CustomWiggle**	plugins	The email-code refusal's head-shake: a wiggle ease whose swings die away.
@gsap/react	2.1.2	The `useGSAP` hook: animations created in a component are cleaned up automatically.
CesiumJS	1.145.0	The 3D globe: terrain, imagery, 3D tiles, camera, markers. Loaded at runtime from `/cesium/Cesium.js`.
three.js	0.186.1	The destination survey's renderer: the terrain point cloud, contours, strata planes, photo panels, route and the post pass (`src/features/destinations/survey/gl.ts`).

Technology	Version	What Longitude uses it for
WebGL2 (raw)	browser API	Three hand-written renderers: the jet transition, the tour-page flyby, the login page's Earth horizon (details in section 9).
Canvas 2D	browser API	The destinations atlas dot map, the route arc label layer, the outro desk globe, and the "ORBIT" word texture on the login page.
View Transitions API	browser API	The jet transition's page swap (the new page is revealed through the wake).
Web Animations API	browser API	Keyframes on the view-transition pseudo-elements.

Data, validation and email

Technology	Version	What Longitude uses it for
PostgreSQL	18 (embedded) / 17 (Docker)	The database: destinations, attractions, tours, itineraries, bookings, users, email-code challenges.
Prisma ORM	7.10.0 (`prisma` , `@prisma/client`)	Schema (`prisma/schema.prisma`), migrations, type-safe queries, the seed. The client is generated into `src/generated/prisma` .
@prisma/adapter-pg + **pg**	7.10.0 / 8.23.0	Prisma's driver adapter over the `pg` Postgres driver (short connection timeout so an absent database is detected fast).
Zod	4.6.5	Validates every API body and query (`src/lib/validation.ts`), turning problems into per-field messages.
Nodemailer	10.0.12	Sends the sign-in code emails over SMTP (`src/server/mail.ts`).

Development tools

Tool	What it's for
ESLint 9 + `eslint-config-next`	Linting, including React's rules for hooks and the React Compiler's purity checks (`npm run lint`).
embedded-postgres	Runs a real Postgres from npm-provided binaries, so no Docker is needed (`npm run db:local`).
Docker Compose	Alternative Postgres (`postgres:17-alpine` , `npm run db:up`).
tsx	Runs the TypeScript seed script.
dotenv	Lets the Prisma CLI read `.env.local` .
Prettier (run with `npx` , width 160)	Formatting; not a project dependency.
Chrome + puppeteer-core (outside the repo)	Used during development to drive the site, capture frames of every animation and measure frame pacing.

---

3. Outside services and data sources

Service	Used for	Where in the code	Needs a key?
Cesium ion	Streams Google Photorealistic 3D Tiles (asset 2275207), Cesium World Terrain, Cesium World Imagery, Cesium OSM Buildings; also its Google-powered geocoder for search.	`src/features/globe/cesium/ionWorld.ts` , `src/server/services/geocoding.ts`	Free token (`NEXT_PUBLIC_CESIUM_ION_TOKEN`)
NASA GIBS	Blue Marble day imagery and VIIRS city lights for the globe from orbit.	`src/features/globe/cesium/scene.ts`	No

Service	Used for	Where in the code	Needs a key?
Open-Meteo geocoding	Place search fallback when there's no ion token or Google key.	`src/server/services/geocoding.ts`	No
OpenStreetMap Nominatim	Reverse geocoding fallback (coordinates → place name).	`src/server/services/geocoding.ts`	No
Google Geocoding / Places (New)	Optional: only if you add your own server key.	`geocoding.ts` , `attractions.ts`	Optional key
Wikipedia API	Attractions near places that aren't in the curated catalog.	`src/server/services/attractions.ts`	No
AWS Terrain Tiles (Mapzen)	The real elevation under each destination, sampled once by `scripts/build-survey.ts` into `public/survey/<slug>.json` . Not called at runtime.	`scripts/build-survey.ts`	No
Wikimedia Commons	All photos, hotlinked. Thumbnails are requested at standard widths 330, 500, 960 or 1280 px (`thumb()` in `src/features/tours/tourData.ts`).	Everywhere photos appear	No
Your SMTP provider	Delivers the 6-digit codes (Gmail, Resend, Brevo, Mailgun, SendGrid, Mailtrap…).	`src/server/mail.ts`	SMTP login

`GET /api/health` reports whether the database is connected, whether the ion token is set and which geocoder is active (`google` , `ion-google` or `open`).

---

4. Architecture

Server and client

- **Server Components** (the default) run only on the server: they read the database through repositories and send finished HTML. Examples: `src/app/tours/page.tsx` , `src/app/account/page.tsx` .
- **Client Components** start with `"use client"` and run in the browser: the globe, the animated sections, the forms. They receive data as props from the server pages, or call the JSON API.
- **Server-only code** lives in `src/server/` and is never imported by client components (secrets stay on the server).

Layers

Layer	Folder	Rule
Pages and API routes	`src/app`	Thin: validate, call a repository or service, render or reply.
Repositories	`src/server/repositories`	The only code that touches the database (`catalog.ts` , `bookings.ts` , `users.ts`).
Services	`src/server/services`	Outside APIs: `geocoding.ts` , `attractions.ts` .
Auth, codes, mail	`src/server/auth.ts` , `otp.ts` , `mail.ts`	Passwords, sessions, email codes, sending email.
Database access	`src/server/db.ts`	One Prisma client; `withDb()` (throws a friendly error if the database is down) and `tryDb()` (returns null so pages can fall back).
Shared helpers	`src/lib`	Types, formatting, validation, geodesy, environment, GSAP setup, per-city themes.

Layer	Folder	Rule
Features	`src/features`	Large interactive pieces (globe, journey, destinations, tours, account, location).

Resilience

The site is built to keep working when parts are missing: without a database the catalog comes from `src/data/catalog.ts`; without a Cesium token the globe uses NASA imagery; without SMTP (in development) codes go to the terminal. `db.ts` remembers an outage for 15 seconds so a missing database doesn't slow every request.

Errors

Every API error has the same shape, produced by `jsonError()` in `src/server/http.ts`:

```
{ "error": { "code": "OTP_INVALID", "message": "That code doesn't match.", "attemptsLeft": 4 } }
```

---

5. Pages and what's on them

Route	File	Main components	Notable techniques
`/`	`src/app/page.tsx`	`JourneyProvider`, `JourneyExperience`, `DestinationDossier`, `DestinationTours`, `DestinationReel`	Pinned CesiumJS globe; ScrollTrigger maps scroll to scenes (Earth → Location → Choose → Route → Arrival → Attractions → Tours); `CameraDirector` damping; HTML markers projected from 3D; great-circle route with a flying plane.
`/destinations`	`src/app/destinations/page.tsx`	`DestinationsAtlas`, `FlightMap`	Pinned Canvas 2D dot-matrix world with flight arcs from IXC; brief hover previews; arrives via the jet transition.

Route	File	Main components	Notable techniques
`/destinations/[slug]`	`src/app/destinations/[slug]/page.tsx`	`DestinationSurvey` (`src/features/destinations/survey`)	The orbital survey: the city's real terrain as a point cloud with contours, flown through by scroll in chapters (acquire, brief, strata, each landmark with its photo decoding on a panel, the route with its elevation profile), then the city's tours rising over the survey; instrument panel, cursor probe reading real elevation, labels on leader lines; Canvas 2D fallback. Static generation per destination.
`/tours`	`src/app/tours/page.tsx`	`ToursHero`, `TourRoute`, `TourPass`, `TourTimetable`, `ToursArrivals`	3D photo wall (CSS 3D, its columns still while the page scrolls); a horizontal route held in place with `position: sticky`, the room changing colour per city; boarding-pass cards with tilt and glare; Flip re-sort; "Surprise me" roulette.
`/tours/[slug]`	`src/app/tours/[slug]/page.tsx`	`TourHero`, `TourBrief`, `TourDeck`, `TourDays`, `TourStay`, `TourPack`, `TourBook`, `TourNext`, `TourBar`	City theme colours; word-by-word scroll reveal; WebGL2 flyby with a wake that reveals a photo, then a fan of prints that take turns; odometer day counter; flipping key card; seat map with rolling fare; boarding transition hand-off.

Route	File	Main components	Notable techniques
`/book/[slug]`	`src/app/book/[slug]/page.tsx`	`BookingCheckin` (with `DepartureCalendar`, `TypedInput`, `Swap`, `Fare`), inside the `/book` layout's `FlapCurtain`	City theme on warm black; a departures board whose rows roll in; four-step check-in that fills a live boarding pass; per-tour session draft; error summary with jump links; keyboard calendar; seat map; booking bar on small screens; traveller count from the seat picker (`?travelers=5`, clamped 1–12); server-side validation and pricing; not indexed.
`/book/confirmation/[reference]`	`…/confirmation/[reference]/page.tsx`	`ConfirmationView`	The curtain opens onto it; the reference on a board of flaps; the pass prints out of a slot; masked phone and email; `.ics` calendar file; print stylesheet; a "database unavailable" view.
`/account`	`src/app/account/page.tsx`	`LoginHero`, `EarthHorizon`, `AuthForms`, `OtpStep`, `BookingLookup`	WebGL2 Earth horizon shader with a sun cycle; boarding-pass form with failure stamps and a split-flap board; the email-code step and its animations.

Route	File	Main components	Notable techniques
`/about`	`src/app/about/page.tsx`	`AboutJourney` (`src/features/about`)	One full-screen stage and one scrolled story in WebGL2 (three.js): the moon comes alive as the Earth, a dive through cloud, dusk above a ray-marched sea of cloud with the story, a satellite among lenses of glass whose beam draws the cloud below the way each lens sees the world (how it works), the descent into the Kullu valley as a living photo, a porthole back to orbit, and a planet you can turn, read and drop a pin on, with the contact details; the policies below. Anchors `#how-it-works`, `#contact`, `#policies` still land.
any unknown URL	`src/app/not-found.tsx`	—	The 404 page.

Shared on every page (`src/app/layout.tsx`): `SiteNav` (with `HudClock`), `SiteFooter`, `FlightTransition`, `TourTransition`, the film grain overlay and a "Skip to content" link.

---

6. API reference

All routes live in `src/app/api/**/route.ts` and speak JSON.

Catalog and places

Method and path	Input	Returns
`GET /api/destinations`	—	`{ destinations }` summaries.
`GET /api/destinations/:slug`	—	`{ destination, packages }`, or `404 NOT_FOUND`.
`GET /api/packages`	`?destination=paris` (optional)	`{ packages }`.
`GET /api/packages/:slug`	—	`{ package }`, or `404 NOT_FOUND`.

Method and path	Input	Returns
`GET /api/places/search`	`?q=manali` (2–80 characters)	`{ results, provider }`: curated destinations first, then geocoder results within 25 km removed as duplicates.
`GET /api/places/reverse`	`?lat=30.73&lng=76.78`	`{ place }`, or `400 BAD_COORDINATES`.
`GET /api/attractions/nearby`	`?lat&lng&name`	Attractions near a non-curated place (Wikipedia or Google Places).
`GET /api/health`	—	Database state, ion token status, active geocoder.

Bookings

Method and path	Input	Returns
`POST /api/bookings`	`{ packageSlug, leadName, email, phone, travelers (1–12), travelDate (≥ 3 days ahead), originCity?, notes? }`	`201 { booking }` with a reference like `LGT-7K3Q-9XA`; attached to the account if signed in. `422 VALIDATION_FAILED`, `404 PACKAGE_NOT_FOUND`.
`POST /api/bookings/lookup`	`{ reference, email }` (a POST body so the email stays out of URLs and logs)	`{ booking }`, or `404 NOT_FOUND`.

Accounts and email codes

Method and path	Input	Returns
`POST /api/auth/register`	`{ name, email, password (8+) }`	`202 { otp }` (a code ticket, see below). `409 EMAIL_TAKEN`, `422`, `429 OTP_RATE_LIMITED`, `502/503 OTP_SEND_FAILED`.
`POST /api/auth/login`	`{ email, password }`	`202 { otp }`. `401 INVALID_CREDENTIALS` (same message whether the email or password is wrong).
`POST /api/auth/otp/verify`	`{ challenge, code }`	`201 { user }` (new account) or `200 { user }` (sign-in), and sets the session cookie. `401 OTP_INVALID` + `attemptsLeft`, `410 OTP_EXPIRED`, `429 OTP_LOCKED`, `404 OTP_NOT_FOUND`, `409 EMAIL_TAKEN`.
`POST /api/auth/otp/resend`	`{ challenge }`	`{ otp }` with a fresh code. `429 OTP_RESEND_TOO_SOON` + `retryAfter`, `429 OTP_RATE_LIMITED`, `404 OTP_NOT_FOUND`.
`POST /api/auth/logout`	—	Clears the session cookie.
`GET /api/auth/session`	—	`{ user }` or `{ user: null }`.

A code **ticket** looks like:

```
{
  "challenge": "vbovLQ42EzGxLc830OYzJzMGgfx8FZcB",
  "email": "you@example.com",
  "purpose": "REGISTER",
  "length": 6,
  "expiresAt": "2026-09-29T10:16:31.002Z",
  "resendAt": "2026-09-29T10:07:01.002Z",
  "attemptsLeft": 5
}
```

In development without SMTP it also carries `devCode` so the flow can be tried without email. Production never includes it.

---

7. Database

Defined in `prisma/schema.prisma`; migrations in `prisma/migrations/`.

Table	Purpose	Key fields
`Destination`	The 8 curated cities (Paris, London, Dubai, Tokyo, New York, Chandigarh, Delhi, Manali)	`slug`, `name`, `country`, coordinates and elevation, `timezone`, `heroImage`, camera framing for the 3D arrival (`cameraHeading`, `cameraPitch`, `cameraRange`)
`Attraction`	Landmarks in each city (38)	`slug`, `destinationId`, coordinates, `focusHeight`, view angles for the camera, `imageUrl`, `bestTime`, `visitDuration`
`TourPackage`	The 16 tours	`slug`, `durationDays`, `priceInr`, `pace`, `groupSize`, `images[]`, `highlights[]`, `activities[]`, `includes[]`, `excludes[]`, hotel fields, `featured`
`ItineraryDay`	A tour's day-by-day plan	`packageId`, `day`, `title`, `description`, `attractionSlugs[]`
`Booking`	A confirmed booking	`reference` (unique), `packageId`, optional `userId`, traveller details, `unitPriceInr`, `taxesInr`, `totalInr`, `status` (`PENDING`, `CONFIRMED`, `CANCELLED`)
`User`	An account	`email` (unique), `name`, `passwordHash`, `emailVerifiedAt`
`OtpChallenge`	One email-code attempt	`id` (random, held by the browser), `email`, `purpose` (`REGISTER` / `LOGIN`), `codeHash`, `payload` (JSON), `attempts`, `sends`, `expiresAt`, `lastSentAt`, `consumedAt`

Migrations so far: `20260926000000_init` (everything except the codes) and `20260929000000_email_otp` (the `OtpChallenge` table, the `OtpPurpose` enum and `User.emailVerifiedAt`).

The seed (`prisma/seed.ts`) copies `src/data/catalog.ts` into the database, so the catalog has one source.

---

8. Accounts, sessions and the email code

Piece	How it works	File
Passwords	`scrypt` with a random 16-byte salt per user, stored as `scrypt$salt$hash`; checked with a constant-time comparison.	`src/server/auth.ts`
Session	A cookie `lgt_session` holding `{ uid, exp }` (base64url) plus an HMAC-SHA256 signature keyed with `AUTH_SECRET`. `httpOnly`, `SameSite=Lax`, `Secure` in production, 30 days. No session table.	`src/server/auth.ts`
Email code	6 digits from `crypto.randomInt`; stored only as `HMAC-SHA256(AUTH_SECRET, "otp:" + challenge + ":" + code)`; 10-minute life, single use, 5 guesses, resend after 30 s (max 5), at most 6 new attempts per address per hour; constant-time check; a wrong guess is counted atomically so two tabs can't exceed the limit; a right code is consumed atomically so it can't be used twice.	`src/server/otp.ts`
Sign-up	Details checked → email not taken → password hashed → code emailed (payload `{ name, passwordHash }`) → right code creates the user with `emailVerifiedAt` set.	`src/app/api/auth/register`, `otp/verify`
Sign-in	Password checked → code emailed (payload `{ userId }`) → right code starts the session and records the address as verified.	`src/app/api/auth/login`, `otp/verify`
Email	HTML (table layout, inline styles, dark card, the code as six boxes) plus a plain-text version. Development without SMTP prints a box in the terminal.	`src/server/mail.ts`
Validation	Zod schemas for every form (`registerSchema`, `loginSchema`, `otpVerifySchema`, `bookingSchema`…).	`src/lib/validation.ts`
Guest lookups	Reference + email in a POST body, never the URL.	`src/app/api/bookings/lookup`

---

9. 3D, graphics and animation

The globe (`src/features/globe`)

- **Loading:** `loadCesium.ts` loads the prebuilt CesiumJS bundle from `/cesium` (copied by `scripts/copy-cesium-assets.mjs`); the component is loaded with `next/dynamic({ ssr: false })` .
- **Looks:** `scene.ts` builds the viewer with NASA Blue Marble and VIIRS night lights and a light that follows the camera; `orbitalField.ts` adds a halo of orbiting specks.
- **The 3D world:** `ionWorld.ts` streams terrain and imagery, then Google Photorealistic 3D Tiles below about 2,400 km (OSM Buildings if those are unavailable). A frame-rate governor lowers detail while the camera moves and restores it when it rests; on a slow GPU it can coarsen the moving detail further, but a still view always gets the full detail (its frame rate doesn't show).
- **Landmarks that are sharp on arrival** (`cityPreload.ts`). A hidden camera is aimed at the shots ahead (the city, then the landmark the camera is heading for and the two after it, framed exactly where a jump to a landmark comes to rest) and Cesium's flight-preload pass fetches their tiles. Shots are prepared at the detail a view settles on, not the coarser one it travels at, so arriving needs no second wave of downloads. The view on screen always comes first: from the moment it asks for tiles until every download it started has landed, the preloading waits (before, it kept filling the connection: the Eiffel Tower's own view needs ~9 MB, but arrived behind ~50 MB for the landmarks after it). The tile cache holds 1 GB, so the prepared landmarks aren't evicted before you get there, and downloads run 24 at a time while the camera rests. Measured on a cold cache (a ~50 Mbit/s connection, Intel UHD): spending ~3 s on each landmark, every one is fully sharp ~0.45 s after the camera stops (the limit of the measurement); clicking straight through with no pause, 0.45–3.3 s.
- **Camera:** `pose.ts` (the pose model), `scrollCamera.ts` (scroll progress → target pose), `director.ts` (the single owner of camera motion, frame-rate-independent damping), `flights.ts` (one-off flights).
- **Route and markers:** `route.ts` draws the great-circle arc; `plane.ts` flies a plane along it; `markerLayer.ts` positions HTML markers over 3D points.
- **Engine:** `engine.ts` wraps it all in an imperative `GlobeEngine` that React drives.

The jet transition (`src/components/transitions`)

`FlightTransition.tsx` intercepts links to `/destinations` and runs a View Transition. `flight/engine.ts` renders, in one WebGL2 canvas, a procedural airliner (`jetGeometry.ts` , about A350-900 proportions: lofted fuselage, NACA-profile wings with winglets, nacelles, livery) with physically based shading (`shaders.ts` : GGX specular, clear coat, an analytic sky for reflections, ACES tone mapping), a soft shadow, navigation lights and strobes, and the edges of the wake whose shape also clips the incoming page. Shaders compile in parallel where the browser supports it, and the engine is warmed up when you hover the link.

The boarding transition

`TourTransition.tsx` intercepts links to a tour: the clicked photo expands from its exact position to full screen with GSAP, the route and tour name rise in, and the new page's hero (the same photo) takes over when it signals it's ready (`lgt:tour-ready` / `lgt:tour-arrive` events). A watchdog ends it after 9 seconds no matter what.

The booking terminal (`src/features/booking`)

- **Type that rolls into place.** The departures board, the clock, the pass and the confirmation are set in plain Geist Mono (tabular figures, uppercase, generous tracking), so they read at a glance. Motion stays light: the board's cells rise out of their own lines, row by row, as the page opens (one GSAP stagger); a value on the pass that changes (dates, seats, the clock) rolls in through `Swap` , a span keyed on its value so a short CSS animation replays; the total counts to its new figure (`Fare`); the confirmation's reference rises a character at a time. The board sits in its own memoised component, and the inputs and calendar are memoised too, so a keystroke re-renders only its field and the pass. (It used to be a split-flap

board: each character a cell of four animated halves, about 2,400 extra elements on the page, and hard to read; it was taken out for that.)

- **Typed input** (`TypedInput.tsx`). The real input sits on top with transparent text, so the caret, selection, autofill and accessibility are all the browser's own. A mirror behind it draws each character as a span that rises out of a soft blur. Characters keep their identity across edits, so one still animating isn't restarted; a paste arrives as a quick cascade; the mirror follows the input's sideways scroll.
- **The curtain** (`curtain/`). It lives in the `/book` layout, so it spans the change of route. `board.ts` draws the board of flaps on one canvas: each flap's turn about its hinge is projected with the screen's perspective and drawn in four slices, so its free edge narrows or widens, and the light it catches as it shuts (brightness 2.2 → 1) is a second, additive pass of the same sprite. `flapWorker.ts` does the drawing in a Web Worker, on an `OffscreenCanvas` handed over as the page loads. The first accelerated canvas on a thread makes a GPU context synchronously (about 185 ms on integrated graphics); the worker pays that while the page loads, and its frames don't wait on the main thread. Browsers that can't hand a canvas over draw it on the page. The sequence: a wave out from the button (at most 960 ms), the words rise in, the next page is pushed once they have settled (so its first render, and the GPU's first paint of it, happen under a still board), a 650 ms hold, then the flaps fall away from the middle. The curtain opens by itself when the route changes (onto error views too), with a 15-second watchdog; with reduced motion a still board crossfades in and out.
- **Performance, as measured** on Intel UHD integrated graphics in Chrome:
  - one canvas instead of about 150 flap elements, each of which had been a layer with its own raster (about 250 ms of GPU stall as the wave started);
  - plain type instead of split-flap cells: 3,148 elements on the booking page before, about 740 after;
  - nothing on the page animates forever: the board's light sweeps across once, the dots blink for a few seconds, the button's "ready" ring pulses three times (by transform and opacity, not by repainting a shadow), so an idle page costs nothing; the endless versions kept the browser drawing every frame;
  - each form step's heading and note rise in as it scrolls into view, rather than the whole step, which would have been lifted onto a layer and back, repainting all its fields mid-scroll;
  - typed characters use `animation-fill-mode: backwards`, because a held `blur(0)` kept every glyph as a filter;
  - the pass tilt moves by `transform`, because animating inherited custom properties restyled everything on the pass each frame.

  Wheel scrolling the booking page at 1908 × 969: p95 frame 12–17 ms (36 ms before), 3–5 frames over 33 ms in a 4-second scroll (20 before). A keystroke reaches the screen in 24–40 ms. Single long frames remain where the GPU first paints newly visible content, and on a first-ever visit while Chrome compiles shaders it hasn't cached yet.

The tour-page flyby (`src/features/tours/detail/flyby.ts`)

The same airliner, drawn side-on crossing the section left to right. `createFlightPath()` computes the path and the wake: a wedge between the two contrails, narrow at the engines and wider the older it is, which becomes the `clip-path` of the photo layer. `createFlyby()` renders the jet, its lights and the vapour edges. On wide screens the section pins and the whole sequence is scrubbed by scroll; on phones it plays once when it comes into view.

The destination survey (`src/features/destinations/survey`)

- `scripts/build-survey.ts` (run ahead of time) picks a square round the city and its landmarks (its side is 1.6 times their spread plus 3 km, kept between 7 and 46 km), fetches the AWS Terrain Tiles covering it at a zoom that gives about 520 tile pixels across, decodes the Terrarium PNGs with a small built-in decoder (elevation = R x 256 + G + B / 256 - 32768 m) and samples a 200 x 200 grid into `public/survey/<slug>.json`.
- `terrain.ts` loads that file (cached per city; a seeded procedural landscape if it fails) and prepares the land: smoothing (heavier for flat cities), a sea mask, relief normalised so a flat city still reads as land, a slope field for lighting, marching-squares contours, and the route through the landmarks with its elevation profile. It converts latitude and longitude into the square through Web Mercator.

- `story.ts` is the choreography, apart from any renderer. Chapters are kinds (`acquire` , `brief` , `strata` , `spot` , `route`); `update(s)` takes the story position (whole numbers are chapters) and fills a `Frame` : the camera (each chapter's pose, flights between them lifted in the middle, narrow screens stepping back until the chapter fits, the outro's slow orbit), how strongly each layer shows, the scan front, the strata spread, the active landmark with its panel's decode step (40 to 1 blocks) and colour, the route's progress, and the cursor probe (a ray marched against the heightfield). Landmark views frame the whole stack from the pin's foot to the top of its photo, eased to the photo's real shape when it arrives.
- `gl.ts` draws a frame with three.js: the land as points lit by slope and height that scan in from the horizon (water shimmers), contour lines that rise and glow where a strata plane cuts them, a fine grid, the strata planes, pins with rings, each landmark's photo on a camera-facing panel whose shader decodes it from blocks (scattered cells first) and turns from grey to colour, the route with a moving head, the sun, faint stars, and a post pass (FXAA, vignette, scanlines, grain, a little chromatic aberration in motion). It asks for the WebGL2 context itself (a missing GPU is a quiet null), handles context loss and restore, and renders under a pixel budget that a frame-rate governor can lower.
- `flat.ts` draws the same frame with Canvas 2D: the points in a few brightness buckets, contours, strata outlines, pins, the route, and the panel pixelated by drawing the cropped photo small and scaling it up.
- `mosaic.ts` decodes a photo into a 2D canvas in steps of blocks (the brief's photo and the tours' thumbnails).
- `DestinationSurvey.tsx` maps scroll (a ScrollTrigger over a sticky stage) to the story position and runs the loop; a second ScrollTrigger feeds the outro as the tours arrive. It crossfades each chapter's words, fills the chapter panel, places the callout beside the landmark's panel where it clears the corner text, keeps the strata labels level with their planes (pushed apart where they crowd), draws the elevation profile with the route, and reads out the probe. Photos come from Wikimedia with CORS and one retry. The canvases are created in code (not by React) so a lost one can be swapped for a fresh one: WebGL first, the 2D renderer while WebGL is away (with two retries on a new canvas). Reduced motion cuts between chapters in place.

The tours page, kept smooth while scrolling (`src/features/tours`)

Measured in headed Chrome on Intel UHD: wheel-scrolling the route used to run p95 18 ms with stalls of 100–480 ms; now p95 12 ms, p99 12–18 ms, and a whole scroll from the top through the route drops two or three frames (43–85 ms), the first time only.

- **The room's colour** fades in on one of two full-screen layers (opacity only), and the few things in its ink follow with a colour transition. It used to be a tween of CSS variables on the section, which restyled every element and repainted every pass on every frame of the fade.
- **The passes' notches** are holes cut with a CSS mask, not circles painted in the room's colour (which made every pass repaint whenever the colour changed); their shadow sits on a pseudo-element behind them.
- **While the route scrolls,** nothing in it takes the pointer, and a pass's tilt measures itself on entering and draws once a frame: passes sliding under a still mouse used to fire hover effects, layout reads and photo changes mid-scroll. Layout is read on refresh, not in the scroll handler.
- **The stage is held with** `position: sticky` in a section as tall as the journey, not ScrollTrigger's pin, which flips it to `position: fixed` as you arrive and made Chrome rebuild and re-raster every pass at once.
- **The hero's wall** fades under a shade layer (fading the 3D plane itself flattened it into an offscreen surface), no longer scales as you scroll, and its columns hold still while the page scrolls or the hero is off screen: drifting under a moving page they kept uncovering photos the GPU hadn't drawn yet.

The login Earth (`src/features/account/earth`)

`earthShader.ts` is a single full-screen fragment shader that ray-casts a planet horizon with atmosphere and a sun that moves through the day; `EarthHorizon.tsx` drives it, drawing the giant "ORBIT" word on a 2D canvas that becomes a texture the shader lights. A third of the bright stars scintillate (a quicker, uneven flicker); the rest breathe slowly. Through the night a Moon rises from behind the limb, in the clear sky between the pass and the word, and climbs out of the frame by dawn: its phase is lit by the sun, its maria and craters come from noise (no extra download), it silvers the night-side clouds a little, and once it has cleared the limb it lights a fine rim on the side of each letter that faces it. At night the word is a deep silhouette cut out of the stars, its upper faces only just touched by moonlight.

The About page (`src/features/about`)

- **One timeline.** `world/story.ts` holds the beats as stretches of scroll progress p (0 → 1 down an 880svh stage): life spreads 0.015–0.25, the dive 0.265–0.345, fog rises over everything by 0.345 (where the world below takes over) and lifts by 0.405, the story 0.39–0.53, the satellite arrives 0.535–0.645, the lenses 0.56–0.78 while night falls (the sun sets 0.6–0.69, night by 0.75), the descent 0.77–0.87 (its fog covers 0.808–0.836, the valley taking over at 0.82), the living valley to 0.905, the porthole back to orbit 0.9–0.962, and the planet, yours to turn, to 1. It also lists the four chapters with the progress each is read at (`rest`, and `calm` for reduced motion), which the anchors and the chapter index (a rail down the right edge) use. The stage fills the screen; the site's bar sits over it with a soft shade.
- **The conductor** (`AboutJourney.tsx`). ScrollTrigger reports the exact progress; one rAF loop damps it (rate 4.2/s) and hands the damped value both to the 3D world and to a paused GSAP timeline one second long (`tl.progress(p)`), so words and world never drift apart. The first frame lands where the page already is, so a reload or a deep link doesn't fly through the story. The loop runs only while the stage is on screen.
- **Set A, space** (`world/engine.ts`, `world/shaders.ts`). One sphere drawn by a shader that blends the Moon (colour, plus relief from the LOLA elevation: bump-mapped, and displaced on the limb) into the Earth (Blue Marble day, darkened sea with a sun glint, Black Marble city lights on the night side, cloud shadows, a warm terminator) across a front that spreads as an even angle from Chandigarh, torn by 3D noise and glowing where it passes. Texture lookups pick their own gradients so the map's seam never shows. A cloud shell sits just above; the air is a shell round the planet where each pixel finds how low its ray passes, so it glows on the limb, warm at the dusk line, and still works from inside it. The camera swings round the planet to Kullu, set on the dusk line, levelling to look at the actual horizon (it dips as the camera comes down), with the sun to its right.
- **Set B, over the cloud.** A dusk sky dome, the sea of cloud over it, and a satellite over both. The cloud is ray-marched at about 1/2.1 resolution (1/2.5 on an integrated GPU, which Chrome on Windows uses even beside a discrete one): a slab of layered value noise from a 64^3 3D texture, dense low down and billowing on top. Each sample is lit by how much the big shape (two octaves) thins towards the sun, so the fine fraying shapes the edges without speckling the light; the surfaces are soft enough for the step size, rays start a fraction of a step in on an even pattern, and the result is smoothed with a small tent filter as it is scaled up. Each octave of noise is kept only as far as it can be resolved: from the angle between neighbouring pixels' rays and how flat a ray meets the deck, each sample knows how big a patch of cloud it stands for, and an octave finer than that fades to its average (the big billows too, right at the horizon). Sampled sparsely, that detail used to come out as a crosshatch in the far cloud; now the far deck is as smooth as the near one. Together these keep the cloud from looking grainy or shimmering as the camera moves. It glows where you look into the sun and hazes into the horizon. The result is tone-mapped and laid over the sky.
- **The satellite** (`loadCraft` / `surveyCraft` in `world/engine.ts`, `CRAFT_*` in `world/shaders.ts`). NASA's Landsat 8 model (public domain), converted once for the web: positions as int16, normals as int8 and the model's own colours per vertex with a material class in alpha (gold foil, amber foil, silver, white paint, dark parts, solar cells), in one meshopt-encoded stream, 79k triangles in 0.8 MB, decoded with three's meshopt decoder. One standard material, extended with `onBeforeCompile`: foil is crinkled by a bump from 3D noise (faded out before a crinkle gets smaller than a few pixels, so it glints rather than sparkles); the solar wing is a mosaic of dark glass cells, each a touch different in tint and gloss, with fine bus bars, laid out along the wing's own axis (found from its vertices when it loads), so the wing shimmers as it turns. Its metal reflects the dusk worked out directly in the shader (the sky, the low sun, the cloud below), softened with roughness, instead of from a pre-blurred environment map. It flies above the deck, so it is simply drawn after the cloud (thin struts stay crisp, with no low-resolution depth pass to lose them). It glides in from the sun's side, slows over the middle of the frame, drifts while the lenses are shown, turns a little with the pointer and nudges when the lens changes, then sweeps off to the left as we sink into the cloud; on a tall screen it stands farther off, between the title and the stacked lenses. Two lights on it blink; a small tag rides beside it. It turns slowly about its body by itself (not with reduced motion) and with the scroll, and it can be spun by hand: the chapter takes the pointer while it shows, a press is the craft's only if it lands on its body (a disc on screen) or its solar wing (the wing's rectangle, projected), and a drag turns it about as far as the wing under the pointer moved, coasting on after release, while a tilt springs back to level. A tap makes the beam flare. The grab cursor shows over it.
- **Nightfall** (`nightAt`, `SUN_ELEV` in `world/engine.ts`; `SKY_FS`, `NIGHT_STARS_*`). Through "How it works" the sun sinks from just above the cloud into it, and everything follows from where it is: the sky's warm band narrows, reddens and gathers

under where the sun went, then fades to a faint airglow under a truly dark sky; the cloud loses the sun as it meets the deck's horizon and takes the moon's light instead (the light uniforms swing from one to the other), so moonlit billows keep their shape; the satellite, higher up, keeps the sun until the Earth's shadow climbs over it (three's light loop is patched so the sun's light is cut below a rising height, and the tag reads "in sunlight", then "in the Earth's shadow"). Then the night: stars that twinkle and dim near the horizon, a faint Milky Way, a shooting star in some 6.5-second slots, a few satellites crossing on their own great circles and fading as they pass into the Earth's shadow, a crescent moon lit from where the sun went down (the Moon texture from the first chapter, with earthshine and a halo), over a moonlit, truly dark deck. The beam and its patterns show brighter against the dark, and the descent's fog is night-dark. The section's resting point (0.7, where `#how-it-works` lands and reduced motion holds) is the moment just after sunset: the satellite still gold against the dimming sky, the moon and the first stars out.

- **The beam.** A cone of soft light from under the satellite, fading where it leaves the craft and as it sinks into the cloud, with pulses running down it. Where it lands, the cloud carries its light, drawn the way the lens that's on sees the world: an imaging swath with a bright line sweeping down it (Engine), tiles lighting up as they stream in (Cities), radar rings spreading out (Terrain). The pattern is worked out once per pixel, where its ray crosses the cloud tops, not at every step of the march (whose jitter would speckle it), with lines kept about a pixel wide; the cloud's upper layers take the light, its depths don't. A change of lens cross-fades the patterns with a flare. Scroll steps through the three; a hovered or focused lens takes over and a clicked one stays (`aria-pressed`). Dashed leader lines run from each lens to its own spot on the footprint, amber for the lens that's on.
- **The lenses' glass, in WebGL.** Behind each card the scene is drawn at half size (sharp) and a quarter (two blur steps), then drawn back inside a rounded rectangle: mostly see-through, bent and split into colour at the rim like thick glass, with a clear lens under the pointer that magnifies 1.45× and an amber edge on the lens that's on. The satellite is drawn into the lenses' copy only when its disc on screen is behind one. The cards themselves are plain DOM buttons; their rectangles are read each frame from layout plus GSAP's transform cache. (`backdrop-filter` over a live canvas re-blurs every frame and was the biggest cost on integrated GPUs.)
- **The cuts.** Fog rises through the frame from the bottom edge, covers everything, and lifts off the top, in one full-screen pass at half size (two layers of noise, the back one slower and darker, lit along the rising crest, drifting up and in). It is a pure function of scroll and slow time, so it never pops or jitters, and while it covers the frame the world behind it isn't drawn at all.
- **Set C, the valley and the way back** (`VALLEY_FS`). The valley photo with a depth map (`valley-depth.webp`: the skyline found as the smoothest bright-to-dark edge across the columns, the ground's depth growing with distance from the valley's vanishing point), so it shifts in depth with the pointer and as we sink into it; mist drifts through the mid distance and light slants across. It then closes like an iris into a porthole of thick glass (refracted and split into colour at the rim, a reflection arc, a warm halo): the picture holds still until the circle fits the screen, then shrinks with it, so it always fills it, and flies to where Kullu is on the planet, which is pulled back from just above Kullu to the hero's framing.
- **The finale.** The planet turns slowly by itself; a drag turns it (with a little inertia). The disc on screen is tracked each frame and a transparent element over it takes the pointer (`touch-action: pan-y`, so a phone still scrolls). A ray from the pointer to the sphere gives latitude and longitude: hovering shows a ring on the ground and a reticle with the coordinates; a tap sends a ring out across the ground, leaves a pin, and a label with the coordinates and "Plan a trip here", which rides round with the planet and fades as it turns away.
- **The finale's day and the Moon** (`drawFinale`, the `orbit` worked out in `layout`). The sun goes round once every 45 s (not with reduced motion) on a great circle chosen on the finale's screen, so it never crosses the words: on a wide screen it comes in at the top between the logo and the links, crosses behind the planet's right side and leaves by the bottom right corner; on a tall one it comes in from the right edge and sets behind the planet. The rest of the way round it's behind us, so the face we see goes from full day, through a shrinking crescent, to night with the cities lit (only the lights: the night map's blue-grey land is kept to a neutral trace), and back. The sun is a card far behind everything (a white disc, a tight glow, a few fine rays), so the planet and the Moon hide it; a full-screen pass adds its flare (a thin streak, a glare, faint ghosts), scaled by how much of it shows past them and flashing as it slips behind a limb or comes out of one. The air round the planet takes the moving sun, so it rings the night side when the sun is behind. The Moon is the planet's own shader with `MOON_ONLY` (relief and regolith, no living front), turned so its near side faces the Earth, going round every 32 s on a tilted ring that swings low on the left (under the title) and passes behind the planet high on the right; it fades in from behind the planet as we pull back to orbit. Its phase follows the sun, with earthshine on its dark face. Each ball shadows

the other (`ballShadow`: a soft-edged cylinder of shadow from a sphere, in view space): the Moon's shadow crosses the ground and the clouds, and in the Earth's shadow the Moon keeps only a faint dark red. The readout says whether we're looking at the day side, the twilight or the night side.

- **Loading and cost.** The words never wait: the engine is a dynamic import after first paint. Textures are decoded as ImageBitmaps off the main thread (2K instead of 4K on phones), uploaded one per frame, and the cloud world's shaders are compiled and its targets touched while you're still in space; the valley photo is decoded ahead. A governor watches the time between frames and lowers the pixel ratio and the cloud resolution below ~50 fps, raising them again with headroom. The canvas stops drawing once nothing is left on it, and a lost GPU context shows the poster until it comes back. Measured at 1908 × 969 in headed Chrome on this laptop (Intel UHD, 165 Hz screen), production build, wheel scrolling the whole story: in a clean run the median frame is 6.1 ms throughout, p95 12 ms at rest, 12–24 ms through the two cuts, 18 ms over the cloud, 24–36 ms with the satellite, its beam, the lenses and nightfall, 12–18 ms through the valley and porthole and 12 ms on the finale's planet (with its day, the sun's flare and the Moon); nothing over 50 ms once the browser's GPU cache is warm (the first pass after Chrome starts can drop a frame or two at the cuts, under the fog). Every pass's shader variants are compiled ahead, including the ones for drawing into the lenses' render targets (no tone mapping, linear output), which once cost a 160 ms stall on the first lens frame. Runs taken while something else drew on the same GPU doubled the median even on the still hero, so compare numbers only from quiet runs.
- **The words' timeline** is run to its end and back once when it's built (and after a resize), because a paused timeline only applies a tween's starting values once the playhead reaches it: without that, the cards and the wordmark showed in their resting places over the story. Masked text that moves by `yPercent` also pins `y` to 0, because GSAP reads a transform it finds (one left by a remount) back as pixels.
- **Accessibility.** Every chapter stays in the document (only the one showing takes the pointer); tabbing into a chapter that isn't showing brings it in. The manifesto is one label, its words hidden from assistive tech. Reduced motion shows only still chapters, crossfading through dark, with the living Earth first.

The email-code animations (`src/components/account/OtpStep.tsx`)

Moment	Animation	Built with
Step appears	Six slots rise in with a 45 ms stagger	GSAP `fromTo`, `expo.out`
Typing	Each digit pops in; the next slot's caret blinks	CSS keyframes on mount, `steps()` blink
Checking	A light sweeps across the slots, one after another (kept on screen at least 480 ms)	CSS keyframes with per-slot delay
Wrong code	The row shakes its head (a wiggle that dies away) and tints red; every slot turns over like a split-flap departures-board tile to a red ×, left to right (top half falls on the hinge, bottom half swings down with a small bounce, each flap shaded as it tilts); a beat later they turn over again to blank; the message decodes in and a "tries" pip flashes red and drains	GSAP CustomWiggle, nested GSAP timelines driving per-slot flap overlays (`flap()` in `OtpStep.tsx`), CSS 3D `rotateX` with `perspective` and `backface-visibility`, `scrambleText()`, CSS transitions for the pips; `navigator.vibrate` on Android
Expired / locked	The tiles turn over to dashes and stay; "Send a new code" becomes the main action; all pips drained	Same flap overlays, left standing until a new code arrives
Right code	Slots fill amber left to right, digits lift, slots slide together into one seal, a ripple, a tick draws itself; then the "cleared" stamp	CSS transition delays + GSAP timeline, SVG `stroke-dashoffset`
Reduced motion	Same states as fades, no movement	`prefers-reduced-motion`

Motion rules followed everywhere

- Animate `transform` and `opacity` (and `clip-path` where needed); avoid layout properties.
- No `backdrop-filter` on scrolling content; no large blurred text.
- Hover effects only for real pointers: `@media (hover: hover) and (pointer: fine)`.
- A reduced-motion version of every animation (GSAP `matchMedia` with `MOTION_QUERIES`).

- Marquees never pause; cursor-follow previews appear only briefly and hide on scroll.
- Heavy work is prepared ahead (shader compile on hover, WebGL built a screen before it's needed) and every multi-step transition has a watchdog.
- Keep the layer count small: many identical moving pieces go on one canvas, `will-change` only while something moves, and no `backface-visibility` where nothing turns past edge-on.
- No filter is left held by a finished animation (`animation-fill-mode: backwards` for entrances).
- Nothing animates forever on a page meant to sit still: an endless animation keeps the browser drawing every frame, on every frame's budget.
- Frosted glass over a live canvas is drawn in WebGL from a small blurred copy, never with `backdrop-filter`.
- A WebGL pass drawn with a flat full-screen camera can't use three's `cameraPosition`: pass the scene camera's position in yourself (the About clouds once marched from the origin, inside the cloud, and came out as fog).
- Transitions that hide a cut are a pure function of scroll (plus slow time), drawn in one pass: sprites streamed past the lens and re-sorted every frame popped and lurched as you scrolled.

---

10. Design system

Colours (`src/app/globals.css`, `:root`)

Token	Value	Role
`--color-space`	`#06070a`	Page background
`--color-ink`, `--color-panel`, `--color-raised`	`#0b0d11`, `#101217`, `#15181e`	Surfaces
`--color-fg`, `--color-dim`, `--color-muted`, `--color-faint`	`#e8eaee` → `#5a606b`	Text, from primary to faint
`--color-signal`	`#e6a36a`	The one accent (amber): buttons, highlights, the plane
`--color-signal-ink`	`#1b120a`	Text on amber
`--color-danger`	`#e5776b`	Errors

Per-city themes (`src/lib/tourThemes.ts`)

Each city recolours its tour sections (every pair is contrast-checked: ink on ground at least 5.8:1).

City	Theme	Ground	Glow
Paris	Rouge, for the city of love	`#a51c30`	`#e23b54`
Tokyo	Sakura, sumi and torii vermilion	`#f3c6cf`	`#f291a8`
London	British racing green	`#1f4a3b`	`#45a077`
Dubai	Desert sand and Gulf teal	`#e7cc98`	`#e6c07a`
New York	Yellow cab	`#f5c518`	`#f5c518`
Delhi	Red sandstone and marigold	`#94401f`	`#e2773f`
Chandigarh	Le Corbusier blue and ochre	`#2c5c99`	`#5f94dc`
Manali	Glacier blue and deodar green	`#c3dce6`	`#8fc8de`

Type, easing and shape

- Fonts: Geist (text), Geist Mono (labels, numbers), Instrument Serif italic (accents only).
- Easing: `--ease-out-expo: cubic-bezier(0.16, 1, 0.3, 1)` for entrances; `--ease-in-out-quart` for on-screen moves; GSAP `expo.out`, `power3.inOut`, `back.out` for moments of delight.
- Shapes: pills for buttons (with the icon in its own circle), 14–26 px radii for cards and passes.
- Visual language: an "orbital travel desk": boarding passes, flight numbers (LG 101…116), airport codes (IXC for Chandigarh, the home base), HUD labels in mono, stamps and departure boards.

---

11. Configuration

Variable	Default	Used by
`NEXT_PUBLIC_CESIUM_ION_TOKEN`	blank	Globe (browser) and ion geocoder (server)
`GOOGLE_MAPS_SERVER_API_KEY`	blank	Optional Google geocoding and Places
`DATABASE_URL`	`postgresql://longitude:longitude@localhost:5433/longitude?schema=public`	Prisma
`AUTH_SECRET`	dev-only fallback	Session signatures, code hashes (required in production)
`SMTP_HOST`, `SMTP_PORT` (587), `SMTP_USER`, `SMTP_PASS`, `SMTP_SECURE`, `MAIL_FROM`	blank	Sending codes

Other configuration files: `next.config.ts` (no dev badge, Postgres packages kept server-side, Wikimedia image hosts, a year-long cache for `/cesium`), `prisma.config.ts` (loads `.env.local` for the Prisma CLI), `docker-compose.yml`, `eslint.config.mjs`, `postcss.config.mjs`, `tsconfig.json` (the `@/` import alias points at `src/`).

---

12. Scripts and everyday workflows

Task	Command
Run the site	`npm run db:local` (terminal 1), `npm run dev` (terminal 2)
First-time database setup	`npm run db:setup`
Change the data model	Edit `prisma/schema.prisma`, run `npm run db:migrate`, give the migration a name
Reset sample data	`npm run db:seed`
Browse the data	`npm run db:studio`
Check before shipping	`npm run typecheck`, `npm run lint`, `npm run build`
Rebuild the destination terrain	`npm run survey:build` (all) or `npm run survey:build paris` (one); after adding a destination or moving its landmarks
See the email code in development	Look in the terminal running `npm run dev` for the "sign-in code" box

---

13. File-by-file map

`src/app` — pages and API

File	Purpose
`layout.tsx`	Root layout: fonts, nav, footer, transitions, grain.
`globals.css`	Design tokens and all component styles.
`page.tsx`	Home: the 3D journey and the sections below it.
`about/page.tsx`	About page.
`account/page.tsx`	Sign-in page when signed out; profile and trips when signed in.
`book/layout.tsx`, `book/[slug]/page.tsx`, `book/confirmation/[reference]/page.tsx`	The booking page and its confirmation; the layout holds the curtain between them.
`destinations/page.tsx`, `destinations/[slug]/page.tsx`	The atlas and each destination.
`tours/page.tsx`, `tours/[slug]/page.tsx`	The tours page and each tour.
`not-found.tsx`	404 page.
`api/**/route.ts`	The JSON API (see section 6).

`src/components`

File	Purpose
`account/AuthForms.tsx`	The boarding-pass sign-in/sign-up form and its success and failure moments.
`account/OtpStep.tsx`	The email-code step: six slots and its animations.
`account/authFailure.ts`	Turns an error into a departure-board message and a stamp.
`account/DeniedStamp.tsx`, `OrbitStamp.tsx`, `FlapBoard.tsx`	The failure stamp, the "cleared" stamp, the split-flap board.
`account/BookingLookup.tsx`	Find a guest booking.
`destinations/DestinationGrid.tsx`	Asymmetric destination grid.
`layout/SiteNav.tsx`, `SiteFooter.tsx`, `HudClock.tsx`, `Logo.tsx`, `ExploreLink.tsx`	Site chrome.
`tours/PackageCard.tsx`	Tour card.
`transitions/FlightTransition.tsx` + `flight/*`	The jet transition and its WebGL2 engine, geometry, shaders and maths.
`transitions/TourTransition.tsx`	The boarding transition.
`ui/PageHeader.tsx`, `ui/Photo.tsx`	Page opening; remote photo with fallback.

`src/features`

Folder	Purpose
`globe/`	CesiumJS engine (`engine.ts`, `GlobeCanvas.tsx`), `cesium/` (loader, scene, ion world, orbital field), `camera/` (pose, director, scroll camera, flights), `markers/`, `route/` (arc, plane, screen helpers).
`journey/`	`JourneyProvider` (state), `JourneyExperience` (the pinned hero), `timeline.ts`, `progressStore.ts`, HUD and scene panels in `components/`, and the sections below the hero in `sections/` (dossier, marquee, story, tours, spreads, outro globe, star sky).

Folder	Purpose
`destinations/`	`DestinationReel`, `destinationService`, the atlas (`DestinationsAtlas`, `FlightMap`, `worldDots`) and the survey (`survey/`: `DestinationSurvey`, `terrain.ts` (heightfield, contours, route), `story.ts` (chapters and camera), `gl.ts` (three.js), `flat.ts` (Canvas 2D), `mosaic.ts` (block decode)).
`tours/`	`tourData.ts` (shape and helpers), `ToursHero`, `TourRoute`, `TourPass`, `TourTimetable`, `ToursArrivals`, and `detail/` (`TourHero`, `TourJourney` with the brief, prints and days, `TourStay` with the key card and pack list, `TourBook` with the seat map, next tour and booking bar, `flyby.ts`).
`booking/`	`BookingCheckin` (the booking page), `ConfirmationView`, `DepartureCalendar`, `TypedInput`, `pass.tsx` (seats, dates, barcode, pass tilt, a shared clock), `bookingData.ts` (the tour, its flight and the departures board) and `curtain/` (`FlapCurtain`, the canvas board `board.ts`, and `flapWorker.ts`). `pass.tsx` also has `Swap` (a value that rolls in when it changes) and `Fare` (a total that counts to its new figure).
`account/`	`LoginHero` and the WebGL2 Earth (`earth/`).
`about/`	`AboutJourney` (the About page's stage, chapters and conductor) and `world/`: `story.ts` (beats and chapters), `engine.ts` (the three.js world: space, the cloud and the satellite, the valley and the finale; glass, fog, picking, loading, quality), `shaders.ts` (planet, air, stars, sky, clouds and the beam's footprint, fog, glass, blur, the satellite's material, beam and lights, the valley and porthole).
`location/`	Browser geolocation and the time-zone fallback origin.

`src/lib`

File	Purpose
`types.ts`	Shared domain types.
`validation.ts`	Zod schemas and field-error flattening.
`format.ts`	Dates, rupees (`formatInr`), `priceBreakdown` (5% GST).
`geo.ts`	Spherical geodesy (distances, great circles).
`airports.ts`	Airport codes and flight maths.
`env.ts`	Environment access (public and server-only, including SMTP).
`gsap.ts`	Registers GSAP plugins once; `MOTION_QUERIES`.
`textFx.ts`	Text scramble effects for HUD copy.
`tourThemes.ts`	Per-city colours.
`bookingRules.ts`	Booking limits, messages and field checks, shared by the form and the server's schema.

`src/server`

File	Purpose
`db.ts`	Prisma client, outage detection, `withDb` / `tryDb` / `databaseStatus`.
`auth.ts`	Password hashing, session cookie.
`otp.ts`	Email codes: issue, resend, verify.
`mail.ts`	Nodemailer transport and the code email.
`http.ts`	JSON error helpers.
`repositories/catalog.ts`, `bookings.ts`, `users.ts`	Database access.
`services/geocoding.ts`, `attractions.ts`	Outside APIs.

Elsewhere

Path	Purpose
`src/data/catalog.ts`	The curated catalog (the seed's source and the offline fallback).
`prisma/schema.prisma`, `prisma/migrations/`, `prisma/seed.ts`	Data model, migrations, seed.
`scripts/copy-cesium-assets.mjs`	Copies CesiumJS into `public/cesium`.
`scripts/local-postgres.mjs`	The no-Docker Postgres.
`scripts/build-survey.ts`	Samples AWS Terrain Tiles into each destination's 200 x 200 heightfield.
`public/survey/*.json`	Those heightfields (elevation as base64 Uint16 with its range, the square's centre and size).
`public/about/`	The About page's assets: Moon colour and elevation, Earth day, night and a packed cloud/water mask, the Landsat 8 satellite (`landsat8.bin`), the graded Kullu valley photo and its depth map, and the no-WebGL poster.

---

14. Glossary

Term	Meaning
App Router	Next.js routing where each folder in `src/app` is a URL and `page.tsx` is its page.
Server / Client Component	React components that run on the server (fetch data, output HTML) or in the browser (interactivity).
Route handler	A `route.ts` file that answers HTTP requests with JSON.
ORM	Object-relational mapper: Prisma turns database rows into typed objects and back.
Migration	A SQL file that changes the database structure, applied in order.
Seed	A script that fills the database with starting data.
OTP	One-time password: here, the 6-digit code emailed at sign-up and sign-in.
HMAC	A keyed hash: proves data came from someone with the secret, and lets us store a code without storing the code.
scrypt	A deliberately slow password-hashing function that makes guessing passwords expensive.
Constant-time comparison	Comparing secrets in a way that takes the same time whether they match or not.
Rate limit	A cap on how often something can happen (codes per address per hour).
Great circle	The shortest route between two points on a sphere, the path aircraft fly.
3D Tiles	A format for streaming huge 3D models (whole cities) in pieces.
WebGL2	The browser's GPU drawing API; shaders are small programs that run on the GPU.
PBR	Physically based rendering: lighting that follows real material physics (GGX, Fresnel).
ACES	A film-industry tone-mapping curve that turns bright HDR light into displayable colour.
View Transitions API	A browser feature that animates between two page states.
ScrollTrigger / scrub / pin	GSAP tools to tie animation to scroll position and to hold a section in place while you scroll.
Flip	"First, Last, Invert, Play": animating elements from their old layout to their new one.
Reduced motion	An operating-system setting; the site honours it with calmer animations.